Apply filters to SQL queries

Project description

As a security professional at a large organization, my role involves investigating security issues to maintain system integrity. In this project, I examined the organization's data within the `employees` and `log_in_attempts` tables to investigate potential security anomalies involving login attempts and employee machines. By applying advanced SQL filters, I successfully retrieved targeted records to isolate suspicious activity and identify specific employee groups requiring system updates.

Retrieve after hours failed login attempts

To investigate potential unauthorized access, I needed to identify failed login attempts that occurred outside of standard business hours (after 18:00). I used the `AND` operator to filter for both the time condition and the failure status simultaneously.
Command used:

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_time > '18:00' AND success = FALSE LIMIT 5;
+----------+----------+------------+------------+---------+----------------+---
------+
| event_id | username | login_date | login_time | country | ip_address     | su
ccess |
+----------+----------+------------+------------+---------+----------------+---
------+
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12 |
    0 |
|       18 | pwashing | 2022-05-11 | 19:28:50   | US      | 192.168.66.142 |
    0 |
|       20 | tshah    | 2022-05-12 | 18:56:36   | MEXICO  | 192.168.109.50 |
    0 |
|       28 | aestrada | 2022-05-09 | 19:28:12   | MEXICO  | 192.168.27.57  |
    0 |
|       34 | drosas   | 2022-05-11 | 21:02:04   | US      | 192.168.45.93  |
    0 |
+----------+----------+------------+------------+---------+----------------+---
------+
5 rows in set (0.039 sec)

MariaDB [organization]>
```

Output/Result:

The query successfully returned only the failed login attempts that occurred after 6 PM. This precise filter is critical for identifying suspicious after hours activity that warrants further security investigation.

Retrieve login attempts on specific dates

The security team needed to review all login activity surrounding a specific suspicious event that occurred over a two-day period. I used the `OR` operator to combine the results from two distinct dates into a single dataset.
Command used:

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08' LIMIT 5;
+----------+----------+------------+------------+---------+-----------------+--
-------+
| event_id | username | login_date | login_time | country | ip_address      | s
uccess |
+----------+----------+------------+------------+---------+-----------------+--
-------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |
     1 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |
     1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |
     0 |
|        8 | bisles   | 2022-05-08 | 01:30:17   | US      | 192.168.119.173 |
     0 |
|       12 | dkot     | 2022-05-08 | 09:11:34   | USA     | 192.168.100.158 |
     1 |
+----------+----------+------------+------------+---------+-----------------+--
-------+
5 rows in set (0.001 sec)

MariaDB [organization]>
```

Output/Result:
The query returned all login attempts from both May 8th and May 9th, providing the complete timeline of user activity needed to investigate the suspicious event without having to run separate queries.

Retrieve login attempts outside of Mexico

To investigate potential unauthorized access from unexpected geographic regions, I needed to filter out all legitimate login attempts originating from Mexico. I used the `NOT` and `LIKE` operators to exclude all countries starting with 'MEX'.
Command used:

```
MariaDB [organization]> SELECT *
    -> FROM log_in_attempts
    -> WHERE NOT country LIKE 'MEX%' LIMIT 5;
+----------+----------+------------+------------+---------+-----------------+--
-------+
| event_id | username | login_date | login_time | country | ip_address      | s
uccess |
+----------+----------+------------+------------+---------+-----------------+--
-------+
|        1 | jrafael  | 2022-05-09 | 04:56:27   | CAN     | 192.168.243.140 |
     1 |
|        2 | apatel   | 2022-05-10 | 20:27:27   | CAN     | 192.168.205.12  |
     0 |
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |
     1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |
     0 |
|        5 | jrafael  | 2022-05-11 | 03:05:59   | CANADA  | 192.168.86.232  |
     0 |
+----------+----------+------------+------------+---------+-----------------+--
-------+
5 rows in set (0.001 sec)

MariaDB [organization]>
```

Output/Result:
The query successfully excluded entries like 'MEX' and 'MEXICO', returning only logins from other regions (like CAN and USA). This demonstrates how to quickly filter out known-safe data to focus on geographic anomalies.

Retrieve employees in Marketing

The IT team needed to update machines for employees in the Marketing department, but only for those located in the East building. I combined an exact match filter with a wildcard pattern filter using the `AND` operator to pinpoint the exact records.
Command used:

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Marketing' AND office LIKE 'East%' LIMIT 5;
+-------------+-------------+----------+------------+----------+
| employee_id | device_id   | username | department | office   |
+-------------+-------------+----------+------------+----------+
|        1000 | a320b137c219 | elarson  | Marketing  | East-170 |
|        1052 | a192b174c940 | jdarosa  | Marketing  | East-195 |
|        1075 | x573y883z772 | fbautist | Marketing  | East-267 |
|        1088 | k8651965m233 | rgosh    | Marketing  | East-157 |
|        1103 | NULL         | randerss | Marketing  | East-460 |
+-------------+-------------+----------+------------+----------+
5 rows in set (0.036 sec)

MariaDB [organization]>
```

Output/Result:
The query returned exactly the Marketing employees located in East building offices (e.g., East-170, East-195). This precision prevents unnecessary updates to employees in other buildings and ensures the right assets are targeted.

Retrieve employees in Finance or Sales

A separate update was required for all employees in either the Finance or Sales departments. I used the `OR` operator to include records from two different departments in a single query.
Command used:

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE department = 'Finance' OR department = 'Sales' LIMIT 5;
+-------------+-------------+----------+------------+-----------+
| employee_id | device_id   | username | department | office    |
+-------------+-------------+----------+------------+-----------+
|        1003 | d394e816f943 | sgilmore | Finance    | South-153 |
|        1007 | h174i497j413 | wjaffrey | Finance    | North-406 |
|        1008 | i858j583k571 | abernard | Finance    | South-170 |
|        1009 | NULL         | lrodriqu | Sales      | South-134 |
|        1010 | k2421212m542 | jlansky  | Finance    | South-109 |
+-------------+-------------+----------+------------+-----------+
5 rows in set (0.000 sec)

MariaDB [organization]>
```

Output/Result:
The query successfully returned a combined list of all Finance and Sales employees, streamlining the process of gathering the required device information for the cross-departmental update.

Retrieve all employees not in IT

The final update had already been completed for the Information Technology department. I needed to generate a list of all employees except those in IT to avoid redundant work. I used the `NOT` operator to exclude this specific department.
Command used:

```
MariaDB [organization]> SELECT *
    -> FROM employees
    -> WHERE NOT department = 'Information Technology' LIMIT 5;
+-------------+--------------+----------+-----------------+-------------+
| employee_id | device_id    | username | department      | office      |
+-------------+--------------+----------+-----------------+-------------+
|        1000 | a320b137c219 | elarson  | Marketing       | East-170    |
|        1001 | b239c825d303 | bmoreno  | Marketing       | Central-276 |
|        1002 | c116d593e558 | tshah    | Human Resources | North-434   |
|        1003 | d394e816f943 | sgilmore | Finance         | South-153   |
|        1004 | e218f877g788 | eraab    | Human Resources | South-127   |
+-------------+--------------+----------+-----------------+-------------+
5 rows in set (0.001 sec)

MariaDB [organization]>
```

Output/Result:
The query returned all employees across the organization while successfully filtering out the IT staff, ensuring the update team only targets the remaining departments that still require the security patch.

Summary

As a security professional, I successfully utilized SQL filtering operators to investigate potential security issues and manage employee machine updates. By applying `AND`, `OR`, and `NOT` operators to the `employees` and `log_in_attempts` tables, I was able to isolate after-hours failed logins, track specific date-based activity, exclude geographic regions, and target specific departments for system maintenance. These SQL filtering skills are essential for efficiently extracting actionable data from large organizational databases to maintain system security and operational efficiency.